DATASANJ
ANALYTICS CONSULTANCY

FREE GUIDE · ROW-LEVEL SECURITY

Who Sees What

A practical guide to Row-Level Security in Power BI

- 20 chapters, one problem each
- DAX and steps to copy
- A go-live checklist
- Four Python checks you can run

Zohal Zahir
Microsoft Fabric & Power BI consultant · DP-600 · PL-300
November 2026

CONTENTS

What's inside

	How to use this guide
01	The bridge model
02	Leaky tables
03	Workspace roles skip RLS
04	Guest users and #EXT#
05	Four ways to say "the current user"
06	Embedded reports and CUSTOMDATA
07	Static or dynamic RLS?
08	Hiding columns: OLS, CLS and masking
09	Connection identity: SSO or fixed identity
10	RLS and report speed
11	Why "% of total" changes per user
12	Manager hierarchies with PATH
13	One person, two roles
14	RLS in one role, OLS in another
15	Storage modes change RLS
16	Does Copilot respect RLS?
17	Dev → Test → Prod
18	6 things RLS doesn't hide
19	Audit who sees what
20	The whole series on one page
	The RLS go-live checklist
	Check it with Python
Py1	Scan the model
Py2	Check the security table
Py3	Audit who can access, and who skips RLS
Py4	Test as each user, automatically

START HERE

How to use this guide

Row-level security (RLS) decides which rows each person sees in a Power BI report. One report, one model, and every person sees only their own region, team or customer.

The DAX for RLS is usually one line. Most problems come from everything around it: relationships, permissions, identities, storage modes and deployments. This guide covers those, one problem per chapter.

Each chapter is one part of the LinkedIn series, with the same order:

- **The problem** you would notice in a real report
- **Why** it happens
- **The fix**, with DAX or steps to copy
- **Check** items to tick before go-live

At the end you'll find:

- **The RLS go-live checklist**: every check from the 19 chapters on two pages
- **Check it with Python**: four free scripts that run many of these checks for you, on your own computer

Rules that come up again and again:

1. RLS only filters people with **Viewer** access. Workspace Admins, Members and Contributors see everything.
2. Roles **add up**. Someone in two roles sees the rows of both.
3. A filter only reaches other tables **through relationships**.
4. Always test as a **real Viewer**, in the storage mode you'll publish, in the stage you'll publish to.

All examples use made-up data. Microsoft changes Power BI and Fabric often, so check the linked Microsoft Learn pages for the latest details.

The bridge model

The problem. Your RLS test passed, and users still see the wrong data.

Why. The DAX is one line, `[Email] = USERPRINCIPALNAME()`, and it only filters the security table. Your relationships decide whether that filter ever reaches the fact table.

If you join the security table straight to a business dimension, both sides repeat regions, so you get a many-to-many relationship. That can cause ambiguous filter paths, slow queries, and results that look right until one user is mapped to two regions.

The fix. Put a small bridge dimension between them, with exactly one row per region:

```
UserRegionSecurity  →  DimRegion  →  DimCustomer  →  FactSales
   (many)              (one)
```

Every hop is now many-to-one or one-to-many. A user with two regions is just two rows in the security table.

The filter starts on the security table, so it has to travel "uphill" to DimRegion. On that one relationship only:

1. Set **Cross filter direction** to **Both**.
2. Tick **Apply security filter in both directions**.

Keep every other relationship single-direction.

```
-- role rule, on the security table
[Email] = USERPRINCIPALNAME()
```

CHECK BEFORE GO-LIVE

- ☐ The security table joins to a dimension with one row per key, not straight to the fact table Py1
- ☐ "Apply security filter in both directions" is ticked on the security → bridge relationship only Py1
- ☐ A user mapped to two regions sees both, and nothing else

PART 2 · BUILD IT

Leaky tables

The problem. Your RLS filters five tables. Your model has seven.

Why. RLS filters one table. It reaches other tables only by travelling along relationships. Any table the filter can't reach is returned in full, to everyone in the role.

The usual two: a **targets** table related only to the date table, and a **KPI summary** table with no relationships at all. Both look harmless in model view. Both show the whole company to a user who should see one region. "View as" passes, because you check the revenue visuals, not the targets card.

The fix. Every table is one of three things:

1. **On the filter path**: related, directly or through other tables, to the table your role filters
2. **Blocked** with `FALSE()` for restricted roles
3. **A leak**

There is no fourth option.

- If the table has a Region column at the right grain, relate it.
- If it doesn't and it's sensitive, block it with `FALSE()`.
- Never invent a key just to force a join.

```
-- on a sensitive table that can't be related
FALSE()
```

Date tables are usually fine to leave unfiltered. Check them anyway if they hold anything sensitive.

CHECK BEFORE GO-LIVE

- ☐ Every table is on the filter path, blocked with FALSE(), or not sensitive Py1
- ☐ "View as" testing includes targets, budgets and summary visuals, not just sales

PART 3 · SHARE IT

Workspace roles skip RLS

The problem. Your RLS is correct and "Test as role" passed. Your finance manager still sees every region.

Why. RLS only applies to people who **read** the model: workspace Viewers, and people who get the report through an app or a share. Workspace **Admin, Member and Contributor** can edit the model, so RLS is skipped for them. "Test as role" won't show you this, because it applies the role to whoever you pick, whatever their real access is.

Build is not Write. Build lets someone create their own report or use Analyze in Excel on your model. RLS still applies to Read, Build and Reshare. It is skipped for Write.

Access	Can do	RLS applies?
Workspace Viewer	Read reports	Yes
Model Build / Reshare	Build reports, share	Yes
Workspace Contributor	Create, edit, publish	**No**
Workspace Member	Contributor + add people	**No**
Workspace Admin	Everything	**No**

The fix. Someone wants to build on your model or share it? Don't change their workspace role.

Semantic model → **Manage permissions** → add their email or security group → tick **Build** (and **Reshare** if needed).

Admin, Member and Contributor are for the people who build the workspace. Everyone else gets Viewer or the app.

CHECK BEFORE GO-LIVE

- ☐ Business users are Viewers or app users, not Contributors or Members Py3
- ☐ People who build on the model have Build permission, not a higher workspace role Py3
- ☐ Nobody outside the build team has Write on the model Py3

PART 4 · SHARE IT

Guest users and #EXT#

The problem. Your investor's email is in the security table. They open the report and see nothing.

Why. For a B2B guest, `USERPRINCIPALNAME()` usually returns the email they were invited with. In some tenants it returns the long form:

```
name_partner.com#EXT#@yourtenant.onmicrosoft.com
```

If that's what comes back, your security row doesn't match, and RLS filters out everything. "Test as role" won't show you this, because it runs as you, not as the guest.

The fix.

1. Put a small card on a test page that shows what Power BI sees:

```
UPN Check = USERPRINCIPALNAME()
```

2. Ask the guest what the card says. Store **that exact value** in the security table.
3. If a guest gets access through a security group, test with that guest. If their rows don't show, add them to the role directly by email.
4. Test with a real guest account before go-live, every time.

CHECK BEFORE GO-LIVE

- ☐ Guest rows in the security table match what the UPN card shows for that guest Py2
- ☐ Guests who get access through a group were tested, or added to the role directly
- ☐ A real guest has opened the report and seen the right rows

Four ways to say "the current user"

The problem. Your role DAX looks right, but your security table matches nothing.

Why. An RLS rule keeps the rows where a column equals the current user. Power BI has four functions for "the current user", and they don't return the same thing. The value the function returns must exactly match a value stored in your model.

Function	Returns	Store in your table	Use it when
`USERPRINCIPALNAME()`	The user's sign-in email (UPN)	Emails	Almost always. The default
`USEROBJECTID()`	The Entra Object ID (a GUID that never changes)	GUIDs, plus an email column for people to read	Emails change often, or guests keep breaking the match
`USERNAME()`	The UPN in the Power BI service; `DOMAIN\user` in Desktop and on-premises Analysis Services	Depends where it runs	Matching Windows accounts on-premises
`CUSTOMDATA()`	A string an embedding app passes in (blank in normal reports)	Tenant or customer IDs	Embedded, multi-tenant apps (Part 6)

```
[Email]      = USERPRINCIPALNAME()
[ObjectId]   = USEROBJECTID()
[CustomerId] = CUSTOMDATA()
```

The fix. Pick the function first. Then build the column that stores exactly what it returns.

`CUSTOMDATA()` doesn't work in role rules on DirectQuery models (Part 15).

CHECK BEFORE GO-LIVE

- ☐ The role uses USERPRINCIPALNAME() unless there's a clear reason not to **Py1**
- ☐ Every value in the security table is a valid, trimmed UPN with no duplicates **Py2**

PART 6 · BUILD IT

Embedded reports and CUSTOMDATA

The problem. One report has to serve hundreds of customers, each seeing only their own data, and none of them are Power BI users.

Why. In a normal report you add people to RLS roles by email. In an embedded app there are no Power BI users to add. So instead of identifying the person, you pass the **tenant**.

How it works.

1. The user signs in to **your** app.
2. Your backend looks up which tenant they belong to, from your own database.
3. It generates the embed token with the role name and `customData` set to that tenant.
4. The report loads, and the role filters every query.

```
-- role TenantFilter
[CustomerId] = CUSTOMDATA()
```

```
EffectiveIdentity in the embed token:
  username:   "<user or app id>"
  roles:      ["TenantFilter"]
  customData: "147"
  datasets:   ["<dataset id>"]
```

The rule that keeps it safe: the user never sends their tenant, only their login. Your backend decides the tenant. If the browser could send it, someone changes 147 to 148 and sees another customer's numbers.

Worth knowing:

- Always pass the identity and role in the token. Without them, a report on an RLS model fails to load (service principal), or shows all data (master user account).
- Opened outside your app, `CUSTOMDATA()` is blank, so the report shows nothing.
- On DirectQuery, `CUSTOMDATA()` isn't supported in role rules. Pass the customer ID as the **username** instead, with `[CustomerId] = USERNAME()`.

CHECK BEFORE GO-LIVE

- ☐ The tenant ID comes from your backend, never from the browser
- ☐ Every embed token includes the role and identity
- ☐ A test with two tenants shows each only its own data

PART 7 · BUILD IT

Static or dynamic RLS?

Say your sales data has a Region column. Alice should see North. Bob should see North and South.

Static RLS: one role per group. Make a North role (`[Region] = "North"`) and a South role (`[Region] = "South"`). Add Alice to North, and Bob to North and South. New region → new role, then republish the model.

Dynamic RLS: one role plus a security table. Make one role, `[Email] = USERPRINCIPALNAME()` , and keep a table of who sees what: one row per person and region. Everyone goes into that one role. The table decides what each person sees. New region → add a row. No new role, no republish.

Email	Region
alice@contoso.com	North
bob@contoso.com	North
bob@contoso.com	South

```
-- static
[Region] = "North"
-- dynamic
[Email] = USERPRINCIPALNAME()
```

When to use which:

- A few groups that rarely change, or two or three fixed audiences → **static**
- Many users, access that changes often, or you never want to republish just to change access → **dynamic**
- External guests or identities that change → **dynamic** (the table absorbs the change)

Static keeps "who sees what" in roles. Dynamic keeps it in a table.

CHECK BEFORE GO-LIVE

- ☐ You chose static or dynamic on purpose, for this audience Py1
- ☐ Someone owns the security table and keeps it current (Part 19)

PART 8 · BUILD IT

Hiding columns: OLS, CLS and masking

RLS hides **rows**. To hide a **column** (salary, email, card number) there are three tools. Which one you need depends on how the person reaches the data.

First, one rule for all of them: they only apply to Viewers. Admin, Member and Contributor see everything.

There are two doors to your data. Security only works on the door the person actually walks through.

Door 1: the semantic model. Guarded by RLS (rows) and **OLS** (columns). People come this way when they open a report or app, build their own report on the model, or use Analyze in Excel.

Door 2: the source (SQL analytics endpoint, Warehouse). Guarded by **CLS** and **dynamic data masking (DDM)**. People come this way with direct SQL, notebooks, other models built on the same warehouse, other tools, and Direct Lake with the user's own identity. OLS isn't on this path.

Tool	Where	What it does
OLS	Semantic model, per role	Hides the column completely. Visuals that use it show an error
CLS	Warehouse / SQL endpoint	Blocks the column at the source
DDM	Warehouse / SQL endpoint	Shows the column with masked values

```
role 'Hide salary'
    modelPermission: read
    tablePermission Employees
        columnPermission Salary = none
```

```
-- CLS: GRANT is a whitelist. Listed columns show; the rest are blocked
GRANT SELECT ON Customers(Name, Email) TO [Sales Group];
-- or, if they already have SELECT on the table
DENY SELECT ON Customers(CreditCard) TO [Sales Group];

-- DDM: keep the column, mask the values
ALTER TABLE Customers ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');
GRANT UNMASK ON Customers TO [Finance Group];
```

Gotchas. DDM is not a security boundary: people can guess values with range queries, so pair it with CLS or RLS. Source security only tells users apart if their own identity reaches the source (Part 9). With Direct Lake on the SQL endpoint, SQL security changes how queries run, so test visuals that use protected columns.

CHECK BEFORE GO-LIVE

- ☐ Sensitive columns are hidden with OLS in the model Py1
- ☐ Sensitive columns are also protected at the source if people can query it directly
- ☐ The row filter and the OLS rule for an audience sit in the same role (Part 14) Py1

PART 9 · BUILD IT

Connection identity: SSO or fixed identity

The problem. Someone changes a model's connection to fix a refresh error. Next morning, restricted users can see salary and national ID.

Why. Your model reaches the data as an identity. There are two options:

- **SSO:** each user's own identity reaches the source, so security at the source (OneLake security, SQL RLS, CLS) can tell users apart.
- **Fixed identity:** one identity reaches the source for everyone. The source can't tell users apart, so per-user security has to live in the model (RLS and OLS).

The trap: switching from SSO to a fixed identity is a reasonable fix for a connection problem. But if that identity has broad access, every user suddenly gets what that identity can see. Nothing at the source changed. The identity reaching it did.

The fix.

- Decide on purpose where per-user security lives: at the source (needs SSO) or in the model (works with a fixed identity).
- A fixed identity is a valid design: stable refresh, and users don't need access to the source. Then RLS and OLS in the model must cover everything.
- Don't change the connection identity on a model you don't own. Treat it as a security change, not a settings tweak.

CHECK BEFORE GO-LIVE

- ☐ You know which identity each model uses to reach its source
- ☐ If it's a fixed identity, the model's RLS and OLS cover every sensitive row and column
- ☐ Changing a connection identity goes through the data owner

PART 10 · TEST IT

RLS and report speed

The problem. RLS is set up. Did you check what it costs your report?

Why. RLS adds a filter to every DAX query, so it can slow visuals down.

Quick check, in Power BI Desktop (Performance Analyzer):

1. Start recording, refresh visuals, and note each visual's DAX query time with no role.
2. Modeling → **View as** → pick the role.
3. Record again. The gap for each visual is what RLS costs.

Deeper check, in DAX Studio:

1. Connect to the model. You connect as an admin, so RLS is off until you turn it on.
2. On the Advanced tab, use **View As**: pick the role, and for dynamic RLS enter a real email from the security table.
3. Turn on **Server Timings**. Clear the cache before each run.
4. Compare storage engine time and query count, with and without the role. A big jump there means expensive relationship joins.

Making it faster:

- Filter dimension tables, not the fact table.
- Use relationships in the rule, not LOOKUPVALUE over big tables.
- Use two-way filtering only on small tables. Security ↔ DimRegion both ways is fine because both tables are tiny.
- Keep the security table small, with integer keys.
- On DirectQuery, tune the source (indexes).
- Only a few fixed audiences? Separate models can beat RLS.

CHECK BEFORE GO-LIVE

- ☐ Main pages were timed with and without each role
- ☐ Two-way filtering is only on the small security relationship Py1

PART 11 · BUILD IT

Why "% of total" changes per user

The problem. Your "% of total" measure means something different for every user.

```
% of Total =
DIVIDE ( [Sales], CALCULATE ( [Sales], ALL ( DimRegion ) ) )
```

North, South and East each sell 100. An admin sees North = 100 / 300 = **33%**. A North user under RLS sees North = 100 / 100 = **100%**.

Why. `ALL`, `ALLEXCEPT` and `REMOVEFILTERS` clear filters from slicers, visuals and rows, but they **can't remove the RLS filter**. So `ALL(DimRegion)` only gets back to the rows that user is allowed to see.

It's not a leak and it's not a bug. It's a security feature: you can't use ALL() to peek at hidden rows. But "% of total" has quietly become "% of what I'm allowed to see".

The fix.

- Label the measure clearly, for example "% of your regions".
- If restricted users really need a company-wide benchmark, build a small summary table without the secured column, and decide on purpose to show it. That's a disclosure decision, not a DAX trick.
- Test with View as, and check the denominator.

Source: SQLBI, "Computing accurate percentages with row-level security in Power BI".

CHECK BEFORE GO-LIVE

- [ ] Measures with ALL/REMOVEFILTERS were tested as a restricted user Py1
- [ ] Any company-wide benchmark shown to restricted users was a deliberate decision

PART 12 · BUILD IT

Manager hierarchies with PATH

The problem. "How do I make sure each manager sees only their own team, and everyone below them?" And when someone changes teams, their access should move with them.

The fix: dynamic RLS on your org chart. One report, one role, for 5 managers or 500.

1. On the Employee table, add the chain of command for every row:

```
Path = PATH ( Employee[EmployeeID], Employee[ManagerID] )
```

2. Add a team key: your own ID if anyone reports to you, otherwise your manager's ID.
3. Create one role, **Team Hierarchy**, that keeps rows whose Path contains the signed-in user's team key:

```
PATHCONTAINS (
    Employee[Path],
    LOOKUPVALUE ( Employee[TeamKey], Employee[Email], USERPRINCIPALNAME () )
)
```

4. Test with **View as**: tick **Other user** and enter the email, **and** tick the role.
5. Move someone to a new team by changing one ManagerID. Their access follows.
6. Publish, then add a **security group** to the role, not individual people.

Watch out:

- PATH functions don't work in role rules on **DirectQuery**. Build a flat manager-to-employee table in the source instead (one row per manager per person below them), with `[ManagerEmail] = USERPRINCIPALNAME()`. That works in every storage mode.
- On **Direct Lake**, build the Path column in the lakehouse, not as a calculated column.
- RLS only applies to Viewers. Test with a real Viewer account.

Starter and finished Power BI projects for this chapter are on GitHub: github.com/Zohal987654321

CHECK BEFORE GO-LIVE

- ☐ Every manager was tested with View as (Other user + role)
- ☐ Moving one person to a new team changed their access as expected
- ☐ DirectQuery or Direct Lake models use a flat manager table, not PATH Py1

PART 13 · SHARE IT

One person, two roles

The problem. Someone is in two RLS roles. You expect them to see less. They see more.

Why. Power BI **adds roles together**. A row shows if **any** of the person's roles allows it.

Example:

- Role 1: `[Product] = "Bikes"`
- Role 2: `[Region] = "East"`

Someone in both sees **all of East, plus Bikes in every region**.

The rules:

1. One role, two filters → **both** must match (AND)
2. Two roles → **either** one is enough (OR)
3. A "no" in one role can't cancel a "yes" in another. A role with `FALSE()` on a table doesn't stop another role from showing it.
4. A role with no rules shows every row.

The fix. Want East Bikes only? Put both filters in **one** role. Give each person **one** role that covers everything they need. People often get roles through security groups, so check group membership too.

Test the real combination: **View as** → tick **every** role the person holds.

CHECK BEFORE GO-LIVE

- ☐ Each audience has one role with all its filters Py1
- ☐ Group memberships were checked for people in more than one role Py3
- ☐ View as was run with all of a person's roles ticked together

PART 14 · SHARE IT

RLS in one role, OLS in another

The problem. A user is in an RLS role and an OLS role at the same time. Their visuals show errors.

Why. Roles add up (Part 13). Together, the two roles would show every row **plus** the hidden column. Power BI blocks that combination, and the error appears when the visual runs its query.

Role	What it does
East team (RLS)	East rows only, Salary visible
Hide salary (OLS)	All rows, no Salary
Both together	Would be all rows + Salary → **blocked**

The fix. Create **one** role for these users with both rules in it:

- RLS: the row filter, for example `[Region] = "East"`
- OLS: in Tabular Editor, Roles → your role → **Table Permissions** → set the Salary column to **None**. (TMDL view in Power BI Desktop works too.)

```
role 'East managers'
    modelPermission: read
    tablePermission Sales = [Region] = "East"
    tablePermission Employees
        columnPermission Salary = none
```

Then move the users into this role and **take them out of the other two**. If they stay in the old roles, the error comes back.

CHECK BEFORE GO-LIVE

- ☐ No one is in an RLS-only role and an OLS-only role at the same time Py1
- ☐ Each audience's row filter and hidden columns are in the same role Py1

Storage modes change RLS

The problem. You switch a model from Import to DirectQuery, and the RLS role that worked yesterday stops working. Same role, same DAX, different storage mode.

	Import	DirectQuery	Direct Lake
Where the role runs	On the imported data	Turned into SQL for every query	On Delta tables in OneLake
DAX in role rules	All DAX works	No PATH functions or CUSTOMDATA	No PATH calculated column (build it in the lakehouse)
Security table changes show	After refresh	Straight away	After the model picks up new data
Watch out	Refresh after access changes	Speed: tune the source	With SSO, users need data access and can query outside the model

DirectQuery fixes: use a flat manager-employee table for hierarchies (Part 12). For embedded reports, pass the customer ID as the username, not CUSTOMDATA (Part 6).

Direct Lake: with SSO, users need read access to the data, so they can query it outside the model. Use a fixed identity with model RLS, or secure the data with OneLake security.

More setups to check:

- **Composite models:** a DirectQuery security table can be cached. Refresh after access changes.
- **DirectQuery to another semantic model / live connection:** the source model's RLS applies. Set and test roles in the source model, and give every sensitive table there a rule.
- **Aggregations:** RLS only on the detail table makes queries skip the aggregation. Filter a dimension related to both.
- **Direct Lake with OneLake security:** users see rows both layers allow. Avoid two-way filters.

A flat security table with a simple `[Email] = USERPRINCIPALNAME()` role works in all three modes.

Always test in the storage mode you'll publish.

CHECK BEFORE GO-LIVE

- ☐ Role rules only use DAX that your storage mode supports Py1
- ☐ You know where security lives for Direct Lake: model, OneLake, or both Py1
- ☐ Testing was done in the storage mode you publish

PART 16 · TEST IT

Does Copilot respect RLS?

Yes. Copilot queries your model as the signed-in user, so their RLS role filters every answer.

The catch: Copilot follows your RLS exactly as it is, gaps included.

A sales manager asks Copilot for last year's targets. No report page ever showed the Targets table. But if that table isn't on the RLS filter path (Part 2), Copilot can answer with the whole company's targets. Report pages hide tables; Copilot can still find them.

Three places to check:

1. **Tables RLS doesn't reach.** Put every table on the filter path, or block it with `FALSE()`.
2. **Admin, Member and Contributor.** RLS doesn't apply to them, so Copilot answers from all the data. Give report users Viewer access.
3. **Fabric data agents on a lakehouse or warehouse.** An agent on a semantic model uses the model's RLS. An agent on the lakehouse uses the lakehouse's security, not the model's. Secure the source too, with OneLake security or SQL RLS.

Before you turn Copilot on: sign in as a Viewer, and ask Copilot for data they shouldn't see. They should get nothing.

CHECK BEFORE GO-LIVE

- [ ] Every table is on the filter path or blocked Py1
- [ ] Report users are Viewers Py3
- [ ] Lakehouses and warehouses used by data agents are secured at the source
- [ ] A Viewer asked Copilot for data they shouldn't see, and got nothing

PART 17 · DEPLOY IT

Dev → Test → Prod

The problem. RLS works in Test. In Prod, the visuals are blank.

Why. Deployment pipelines move your roles, but not the people in them.

Travels to the next stage	Stays behind
Role definitions and their DAX rules	Role members
Report pages and visuals	Workspace access
Data source settings and parameters	Data, including the security table
	Credentials and refresh schedule

So on the first deployment, Prod's roles have no members. Users who aren't in any role see no data.

Check in Prod after you deploy:

- **Refresh the model.** The security table only has rows after a refresh.
- **Direct Lake models don't rebind to Prod on their own.** On the SQL endpoint, add a data source rule once. On OneLake, check the model's lineage after every deployment and rebind if needed.
- **Give report users Viewer access.** Members and Contributors skip RLS.

The habit: add **security groups** as role members, once in each stage. Later deployments keep them, because anything that isn't copied stays as it was in the target stage.

CHECK BEFORE GO-LIVE

- ☐ Each stage's roles have their security groups
- ☐ Prod was refreshed after the first deployment
- ☐ Prod models read Prod data (check lineage)
- ☐ The same user tests pass in Test and Prod Py4

PART 18 · SHARE IT

6 things RLS doesn't hide

RLS filters rows **inside the model**. Anything saved in the report, hidden from view, or copied out needs its own fix.

1. **Saved slicer and filter selections.** A saved selection can show a name or value. Clear sensitive selections before you publish.
2. **Typed titles and text boxes.** Typed text isn't data, so RLS can't filter it. Keep names and numbers out; use measures for dynamic titles.
3. **Hidden columns.** Hiding a column isn't security. People with Build permission can see it. Use OLS (Part 8).
4. **App audiences.** Audiences choose which reports appear. RLS still decides which data shows.
5. **Exported files.** RLS filters the export, so the file holds only that person's rows. But if they forward it, whoever opens it sees those rows. Use sensitivity labels so protection travels with Excel, PDF and PowerPoint exports.
6. **Publish to web.** It makes a report public to the whole internet, and it doesn't support RLS. Ask your admin to restrict who can use it.

The rule: secure the data in the model. Treat everything in the report as visible to anyone who can open it.

CHECK BEFORE GO-LIVE

- [ ] No sensitive values in saved slicers, titles, page names or text boxes
- [ ] Sensitive columns use OLS, not just "hide" Py1
- [ ] Sensitivity labels are applied to models with sensitive data
- [ ] Publish to web is restricted in your tenant

PART 19 · AUDIT IT

Audit who sees what

The problem. You set up RLS months ago. Since then, people have joined, left and changed teams. Is it still right?

An RLS audit checks five layers.

Layer	Question	Where to look
1. Workspace roles	Who skips RLS?	Workspace → Manage access Py3
2. Model permissions	Who can build on, or write to, the model?	Semantic model → Manage permissions Py3
3. Role rules	Does every role have a clear rule?	Manage roles, TMDL view or Tabular Editor Py1
4. Role members	Who is in each role?	Semantic model → Security, then each group in Entra
5. Security table	Who does the table give access to?	Query it Py2

Look for:

- Admin, Member or Contributor given to people who only read reports
- Build given to people who don't build reports
- A role with no rules (it shows every row)
- Individual people as role members instead of groups
- Leavers, and people who changed teams, still in groups or the security table

The habit: do this every month, and save the results so you can compare with last month. The Python checks in this guide make most of it a few commands.

CHECK BEFORE GO-LIVE

- ☐ Monthly audit done and saved Py1 Py2 Py3
- ☐ Leavers removed from groups and the security table Py2 Py3

PART 20 · SUMMARY

The whole series on one page

1. Build it. Static roles for a few fixed groups. Dynamic RLS (one rule and a security table) for many people. A filter only reaches other tables through relationships, so every table is on the path, blocked, or a leak.

2. Share and test it. RLS applies to Viewers. Admin, Member and Contributor see all the data. Use security groups. Test with View as, and with a real Viewer and a real guest.

3. One person, two roles. Roles add up. Someone in two roles sees the rows of both. A role with no rules shows everything. RLS and OLS split across two roles cause errors.

4. Storage modes. DirectQuery can't use PATH or CUSTOMDATA in role rules. Direct Lake needs a decision about identity and where security lives. Live connections use the source model's roles.

5. Dev → Test → Prod. Deployments move the roles, but not the people in them. Add your groups in each stage, refresh, and check Prod reads Prod data.

6. Beyond the model. Copilot only uses data the user is allowed to see, gaps included. RLS doesn't hide saved slicers, typed text or forwarded files. Audit who sees what every month.

PRINT IT. TICK IT.

The RLS go-live checklist

Every check from the 19 chapters, grouped by when you do it. Items marked Py1–Py4 can be checked by the free Python scripts in the next chapter.

Build it

- [ ] The security table joins to a dimension with one row per key, not straight to the fact table Py1 Part 1
- [ ] "Apply security filter in both directions" is ticked on the security → bridge relationship only Py1 Part 1
- [ ] A user mapped to two regions sees both, and nothing else Part 1
- [ ] Every table is on the filter path, blocked with FALSE(), or not sensitive Py1 Part 2
- [ ] "View as" testing includes targets, budgets and summary visuals, not just sales Part 2
- [ ] The role uses USERPRINCIPALNAME() unless there's a clear reason not to Py1 Part 5
- [ ] Every value in the security table is a valid, trimmed UPN with no duplicates Py2 Part 5
- [ ] The tenant ID comes from your backend, never from the browser Part 6
- [ ] Every embed token includes the role and identity Part 6
- [ ] A test with two tenants shows each only its own data Part 6
- [ ] You chose static or dynamic on purpose, for this audience Py1 Part 7
- [ ] Someone owns the security table and keeps it current (Part 19) Part 7
- [ ] Sensitive columns are hidden with OLS in the model Py1 Part 8
- [ ] Sensitive columns are also protected at the source if people can query it directly Part 8
- [ ] The row filter and the OLS rule for an audience sit in the same role (Part 14) Py1 Part 8
- [ ] You know which identity each model uses to reach its source Part 9
- [ ] If it's a fixed identity, the model's RLS and OLS cover every sensitive row and column Part 9
- [ ] Changing a connection identity goes through the data owner Part 9
- [ ] Measures with ALL/REMOVEFILTERS were tested as a restricted user Py1 Part 11
- [ ] Any company-wide benchmark shown to restricted users was a deliberate decision Part 11
- [ ] Every manager was tested with View as (Other user + role) Part 12
- [ ] Moving one person to a new team changed their access as expected Part 12
- [ ] DirectQuery or Direct Lake models use a flat manager table, not PATH Py1 Part 12
- [ ] Role rules only use DAX that your storage mode supports Py1 Part 15
- [ ] You know where security lives for Direct Lake: model, OneLake, or both Py1 Part 15
- [ ] Testing was done in the storage mode you publish Part 15

Share it

- [ ] Business users are Viewers or app users, not Contributors or Members **Py3** Part 3
- [ ] People who build on the model have Build permission, not a higher workspace role **Py3** Part 3
- [ ] Nobody outside the build team has Write on the model **Py3** Part 3
- [ ] Guest rows in the security table match what the UPN card shows for that guest **Py2** Part 4
- [ ] Guests who get access through a group were tested, or added to the role directly Part 4
- [ ] A real guest has opened the report and seen the right rows Part 4
- [ ] Each audience has one role with all its filters **Py1** Part 13
- [ ] Group memberships were checked for people in more than one role **Py3** Part 13
- [ ] View as was run with all of a person's roles ticked together Part 13
- [ ] No one is in an RLS-only role and an OLS-only role at the same time **Py1** Part 14
- [ ] Each audience's row filter and hidden columns are in the same role **Py1** Part 14
- [ ] No sensitive values in saved slicers, titles, page names or text boxes Part 18
- [ ] Sensitive columns use OLS, not just "hide" **Py1** Part 18
- [ ] Sensitivity labels are applied to models with sensitive data Part 18
- [ ] Publish to web is restricted in your tenant Part 18

Test it

- [ ] Main pages were timed with and without each role Part 10
- [ ] Two-way filtering is only on the small security relationship **Py1** Part 10
- [ ] Every table is on the filter path or blocked **Py1** Part 16
- [ ] Report users are Viewers **Py3** Part 16
- [ ] Lakehouses and warehouses used by data agents are secured at the source Part 16
- [ ] A Viewer asked Copilot for data they shouldn't see, and got nothing Part 16

Deploy it

- [ ] Each stage's roles have their security groups Part 17
- [ ] Prod was refreshed after the first deployment Part 17
- [ ] Prod models read Prod data (check lineage) Part 17
- [ ] The same user tests pass in Test and Prod **Py4** Part 17

Audit it

- [ ] Monthly audit done and saved **Py1** **Py2** **Py3** Part 19
- [ ] Leavers removed from groups and the security table **Py2** **Py3** Part 19

CHECK IT WITH PYTHON

Check it with Python

Many items on the checklist can be checked by a script instead of by eye. These four free Python scripts do that. They run on your own computer, and they only **read**: none of them change your model, workspace or data.

Script	Checks	Chapters	Sign-in
Py1 `check_1_scan_model.py`	Roles, leaky tables, filter direction, identity functions, RLS + OLS clashes, storage modes	1, 2, 5, 7, 11–15, 19	None. Reads your PBIP files
Py2 `check_2_security_table.py`	Duplicate, blank, mistyped and guest emails; keys missing from the dimension	1, 4, 5, 19	None. Reads a CSV or Excel file
Py3 `check_3_access_audit.py`	Who can open the workspace, and who bypasses RLS	3, 4, 13, 16, 17, 19	Power BI, read only
Py4 `check_4_test_as_users.py`	Runs a query as each test user and compares with what you expect	1, 12, 17	Power BI, read only

Download: the full scripts, a sample model with planted mistakes, and tests are in the RLS Health Check kit on datasanj.com (link on the guide's page).

Set up once

Install Python 3.10 or later. Open a terminal in the kit folder:

```
python -m venv .venv
.venv\Scripts\activate          # Windows  (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt
```

Try it on the sample first. You should see 4 high findings from Py1 and 6 issues from Py2.

CHECK IT WITH PYTHON

Py1: Scan the model

What it does. Reads your semantic model's TMDL files (or a model.bim), finds every role, and follows the relationships to work out which tables each role really filters. Then it flags the mistakes from this guide.

Run it. Save your report as a Power BI Project (File → Save as → `.pbip`) with the TMDL format turned on, then point the script at the `.SemanticModel` folder:

```
python check_1_scan_model.py "C:\Reports\Sales.SemanticModel" --out scan.md
```

Example output (sample model):

```
## Findings: 4 high, 1 medium, 8 info
[HIGH] (Part 2) Role 'Regional Sales': table 'Targets' is not filtered.
       Users in this role see ALL its rows. Relate it to a filtered
       table, or add FALSE() if it is sensitive.
[HIGH] (Part 2) Role 'Regional Sales': table 'KPI_Summary' is not filtered.
[HIGH] (Part 14) RLS-only roles ['North Static', 'Regional Sales'] and
       OLS-only roles ['Hide Salary'] both exist. Anyone in one of each
       gets visual errors. Put both rules in one role for that audience.
[MEDIUM] (Part 5) Role 'Regional Sales' uses USERNAME(). ...
[INFO] (Part 11) Measure [% of All Regions] uses ALL/REMOVEFILTERS. ...
```

The key idea in the code. A security filter always flows from the "one" side to the "many" side of a relationship. It only flows back the other way when the relationship is two-way **and** "Apply security filter in both directions" is on. The script walks those paths from each role's filtered tables:

```
def security_edges(rel):
    """Directions a security filter can travel across one relationship."""
    if not rel["active"]:
        return []
    one_to_one = rel["fromCard"] == "one" and rel["toCard"] == "one"
    both = rel["cross"] == "bothDirections" and rel["security"] == "bothDirections"
    edges = [(rel["to"], rel["from"])]          # one side -> many side
    if both or one_to_one:
        edges.append((rel["from"], rel["to"]))  # many side -> one side
    return edges
```

Any table with data that no path reaches, and that the role doesn't block with `FALSE()`, is reported as a possible leak.

Impact. Leaky tables (Part 2) and broken filter direction (Part 1) are the two mistakes "View as" usually misses, because you only check the visuals you expect to change. The scan checks every table, for every role, in seconds.

Limits. It reads design only. It can't see your data or who is inside a security group. It treats date tables as usually safe.

CHECK IT WITH PYTHON

Py2: Check the security table

What it does. Dynamic RLS matches the signed-in user's email to a row in your security table. If an email is mistyped, has a trailing space or the wrong case, that person silently sees nothing. Py2 finds those rows before your users do.

Run it. Export the security table to CSV or Excel. Add the dimension file to check that every key exists:

```
python check_2_security_table.py security.xlsx --email UserEmail --key RegionKey \
       --dimension regions.csv --out issues.csv
```

Example output (sample data):

```
Rows: 7 | Distinct users: 6 | Issues: 6
Row 2 (Part 1) 'alice@contoso.com': Listed 2 times for RegionKey=1 (rows [2, 3])
Row 3 (Part 1) 'Alice@Contoso.com ': Extra spaces around the email
Row 5 (Part 4) 'carol_fabrikam.com#EXT#@contoso.onmicrosoft.com': Guest UPN
Row 6 (Part 5) 'dave@contoso': Doesn't look like a valid UPN/email
Row 7 (Part 1) 'erin@contoso.com': Blank RegionKey: this user matches no rows
Row 8 (Part 1) 'frank@contoso.com': RegionKey=9 is not in the dimension
```

The key idea in the code. Compare emails the way a person would, trimmed and lower-case, so copies that look different to a computer are caught:

```
clean = df[email_col].str.strip().str.lower()
dup = df.assign(_e=clean, _k=df[key_col].astype(str).str.strip())
for (email, key), group in dup[dup._e != ""].groupby(["_e", "_k"]):
    if len(group) > 1:
        ...  # same person, same key, written more than once
```

Impact. Each problem row is one person who opens the report and sees blank visuals, or a guest who can't get in. Running Py2 before each refresh turns support tickets into a one-minute check.

CHECK IT WITH PYTHON

Py3: Audit who can access, and who skips RLS

What it does. Lists everyone with access to a workspace, and marks who **bypasses RLS**: workspace Admins, Members and Contributors, and (with `--admin`) anyone with Write permission on a semantic model. It also flags guests, apps, people added one by one, and Viewers missing from your security table.

Run it. Sign in with `az login` first, or the script opens a browser sign-in:

```
python check_3_access_audit.py --workspace "Sales - Prod" \
       --security security.csv --email UserEmail --out access.csv
```

Add `--admin` if you are a Fabric admin, to include semantic model permissions.

Example output (made-up data):

```
3 of 7 access entries bypass RLS (see Part 3).
  BYPASS  Admin             User  z@contoso.com    (Workspace: Sales Reporting)
  BYPASS  Contributor       User  bob@contoso.com  (Workspace: Sales Reporting)
  BYPASS  ReadWriteExplore  User  dan@contoso.com  (Semantic model: Sales)
2 Viewer(s) added one by one. Groups are easier to keep the same in Dev, Test and Prod (Part 17).
1 guest account(s) have access. Test RLS with a real guest (Part 4).
1 Viewer(s) are not in the security table, so they will see no data
Groups have access; this script can't see who is inside them. Check group members in Entra.
```

The key idea in the code. One REST call returns each person's workspace role, and three of the roles mean "RLS doesn't apply":

```
BYPASS_ROLES = {"Admin", "Member", "Contributor"}

for u in get(f"/groups/{ws['id']}/users"):
    role = u.get("groupUserAccessRight", "None")
    rows.append({..., "access": role, "bypasses_rls": role in BYPASS_ROLES})
```

For model permissions, any access right that includes `Write` bypasses RLS. `Explore` (Build) and `Reshare` don't.

Impact. This is layers 1 and 2 of the monthly audit (Part 19), saved as a CSV you can compare with last month. It catches the most common reason a correct RLS role still shows everything.

Needs. Access to the workspace that lets you see its Manage access list (usually Admin or Member). Fabric admin rights for `--admin`.

CHECK IT WITH PYTHON

Py4: Test as each user, automatically

What it does. "View as" tests one person at a time, by hand. Py4 runs a DAX query against the **published** model **as** each test user (impersonation), and compares the result with what you expect. Run the same test file after every deployment to Test and Prod.

Write a test file (`tests.csv`), one row per test. `expected` can be a number, `>0` , `0` , or `blank` (no rows):

```
user,dax,expected
bob@contoso.com,"EVALUATE ROW(""v"", [Total Sales])",11400
carol@contoso.com,"EVALUATE ROW(""v"", COUNTROWS(Sales))",>0
guest@partner.com,"EVALUATE ROW(""v"", COUNTROWS(Sales))",blank
```

Run it:

```
python check_4_test_as_users.py --workspace "Sales - Test" --model "Sales" --tests tests.csv
```

Example output (made-up data):

```
             user expected       got result
  bob@contoso.com    11400     11400   PASS
carol@contoso.com     5000      3700   FAIL
guest@partner.com    blank (no rows)   PASS

2 passed, 1 failed.
```

The key idea in the code. The Execute Queries REST API accepts the email of the person to impersonate. If the model has RLS, their role filters the answer:

```
body = {"queries": [{"query": dax}],
        "serializerSettings": {"includeNulls": True},
        "impersonatedUserName": user}
r = session.post(f"{API}/groups/{ws_id}/datasets/{ds_id}/executeQueries", json=body)
```

Impact. The same tests run in seconds in every stage, so "RLS works in Test, but Prod is blank" (Part 17) shows up as a FAIL before users see it. The script exits with an error code when a test fails, so it can run in a DevOps pipeline after each deployment.

Needs. The tenant setting *Dataset Execute Queries REST API* turned on, and Build permission on the model. Models with RLS can't be queried with a service principal, so sign in as a person. Limit: 120 queries a minute per user.

Need RLS checked on a real model?

DATASANJ designs, builds and reviews Microsoft Fabric and Power BI models for small and mid-size businesses, inside your own Microsoft tenant. Fixed scope, and you keep everything.

www.datasanj.com · d_zohal@datasanj.com
linkedin.com/in/dr-zohal-zahir-46128697 · github.com/Zohal987654321

© 2026 Zohal Zahir, DATASANJ Analytics Consultancy. Free to share in full with credit. All examples use made-up data. Power BI and Fabric change often; check Microsoft Learn for the latest details. Not affiliated with or endorsed by Microsoft.